DATA FORGE 2026 · PATHWAY & RIME TRACKS

Rime Voice-Native vs. Pathway BDH Frontier Explainer

Deep-dive research, track-selection judgment, and production-ready solution blueprints for both hackathon tracks — executed against the three-prompt brief.

Prepared for: Data Forge 2026 team • Event window: 21 Aug – 12 Sep 2026 (online, Unstop) • Sponsors: Pathway & Rime • Prize pool: ₹3,00,000
Document generated: 6 Sep 2026 • Source: `Untitled document.pdf` (Prompt 1: Research, Prompt 2: Track Selection, Prompt 3: Solution Assembly)

Contents

Part 1 — Research (Prompt 1)	Track A & Track B technical deep dive
Part 2 — Track Selection (Prompt 2)	Scoring, failure modes, decision tree, recommendation
Part 3 — Solution Assembly (Prompt 3)	Approach 1 (Rime) and Approach 2 (Pathway) blueprints
Appendix	One-page Track B concept briefing (ready to submit draft)

This report treats the two tracks as mutually exclusive hackathon submissions and answers the brief's three prompts in sequence: research both tracks, judge which to pick, then assemble a complete build plan for both so the choice in Part 2 is backed by a real blueprint either way.

PROMPT 1 · RESEARCH

Track A Research — Rime Voice-Native Product

1. Three personas where removing voice breaks the product

Persona & workflow	Why voice is load-bearing, not decorative	Failure mode if voice is removed
Field safety inspector doing hands-busy, gloves-on equipment walk-throughs (industrial, construction, utility)	Hands and eyes are occupied with the equipment; the inspector must log defects, dictate torque readings, and query spec limits mid-task without breaking physical contact with the job.	A screen forces the inspector to stop, remove gloves/PPE, type, and resume — adding minutes per checkpoint and creating a safety incentive to skip logging entirely.
Emergency/dispatch telephony agent triaging incoming calls (roadside assistance, non-emergency 311, logistics dispatch)	The customer's only channel *is* a phone call over PSTN; there is no app, no screen, and often a distressed or moving caller who cannot type.	There is no fallback UI — without voice the product simply cannot receive the request. This is the purest "voice or nothing" case.
Real-time debate / interview coach giving live rebuttal or delivery feedback while a person is speaking	The value is coaching *during* continuous speech (pacing, filler words, argument gaps) — the interaction is inherently full-duplex and time-critical; typed feedback arrives too late to be actionable.	A chat window converts a live skill (speaking under pressure) into an asynchronous review exercise, destroying the core value proposition.

All three share the judging-relevant property the rubric rewards under "Problem & necessity of voice": each persona has zero graceful degradation to a text UI — voice is the interface, not a convenience layer on top of one.

2. End-to-end technical pipeline

LiveKit Agents is the orchestration layer: it manages the WebRTC room, ingests the caller's audio track, and runs the agent's event loop (VAD → STT → LLM → TTS) as a single supervised process per session, which is what makes barge-in and cancellation tractable in one place rather than across disconnected services.

```
Caller mic (WebRTC/SIP trunk)
   │
   ▼
[VAD: Silero] ── continuous, runs even while agent is speaking
   │  speech-start / speech-end events
   ▼
[STT: Deepgram Nova streaming | Whisper streaming] ── partial + final transcripts, low-latency interim hypotheses
```

```
   │  final transcript → turn boundary (endpointing / turn-detector model)
   ▼
[LLM orchestrator] ── streaming token generation, function-calling fences for tool calls
   │  emits sentence-chunked text as soon as a clause is safe to speak
   ▼
[Rime TTS, WebSocket streaming, use_websocket=True] ── converts text chunks to audio
   │  streams audio frames + word-level timestamps back over the same socket
   ▼
LiveKit audio track → caller speaker
   │
   └── VAD fires during agent speech → interruption handler cancels in-flight
       LLM generation + purges queued Rime audio frames → rollback to listening state
```

Two properties of this pipeline matter for the rubric's "hard voice engineering" line: (1) STT, LLM, and TTS are all *streaming*, so the agent starts speaking on the first safe clause instead of waiting for a full LLM response, and (2) Rime's WebSocket mode returns word-aligned timestamps, which is the primitive that makes precise mid-word audio cancellation possible instead of only stopping at chunk boundaries.

3. Engineering the acceptance tests

(a) Sub-400ms time-to-first-audio (TTFA)

- **Definition:** TTFA = timestamp of the first audio sample reaching the LiveKit output track minus the timestamp of the user's end-of-turn (VAD speech-end / endpointer decision).
- **Instrumentation:** tag every pipeline stage with a monotonic timestamp (STT-final, LLM-first-token, TTS-first-byte, track-publish) and log them as structured events per turn; TTFA is derivable from the log without relying on wall-clock guesses.
- **Test harness:** drive the agent with a pre-recorded WAV over a synthetic participant (not a live mic) so the run is repeatable; run N=20+ turns and report p50/p90 TTFA, not a single cherry-picked run.
- **Budget allocation to hit <400ms:** STT partial-final flush ~80-120ms, LLM first-sentence token ~120-180ms (favor small/fast first-token models or speculative "filler" acknowledgement), Rime WebSocket first-audio-byte ~80-120ms. Each stage must publish its own latency so a regression is attributable, not just visible in the aggregate.

(b) Audio cancellation / state rollback on interruption

- **Trigger:** VAD detects caller speech while agent audio is actively playing (post the ~1s adaptive-interruption cooldown window LiveKit recommends, to avoid false positives on backchannel "mm-hmm").
- **Assertion 1 (audio stop latency):** from VAD-fire timestamp to the last audio frame published to the track must be <200ms; verified by capturing the outgoing track's frame timestamps in a test double instead of trusting player-side silence.
- **Assertion 2 (no stale payload leakage):** after cancellation, assert the Rime WebSocket send queue and any buffered LLM continuation are discarded — replay the same interrupted turn and assert zero audio frames arrive with a sequence id from the cancelled turn.
- **Assertion 3 (state rollback):** the agent's conversation state machine must return to `listening` with the interrupted turn marked `aborted` (not `completed`) in the transcript log, so the next LLM

call is not conditioned on a reply the user never fully heard.

Track B Research — Pathway Frontier Explainer (BDH)

1. BDH vs. BDH-CQ fundamentals

BDH (Dragon Hatchling), from Pathway's paper "*The Dragon Hatchling: The Missing Link between the Transformer and Models of the Brain*" (arXiv:2509.26507), reframes attention as a biologically-plausible alternative: a scale-free network of neuron-like particles with explicit excitatory/inhibitory circuits. The paper derives BDH directly from the attention equations, showing that the retrieval computation standard attention performs can be re-expressed as **synaptic memory** stored in a high-dimensional, sparse neuron space.

- **Hebbian synaptic updates:** instead of a fixed weight matrix trained purely by backprop, BDH's memory is written incrementally by outer-product-style updates — "neurons that fire together wire together" — giving a pedagogical closed form $W_t = W_{t-1} + \eta\, x_t\, y_t^T$, where each token's activation reinforces the synapse between co-active units.
- **Sparse, non-negative activations:** unit activations are rectified (non-negative) and encouraged to be sparse, which the paper connects to emergent modularity and *monosemantic* synapses — individual connections that stay interpretable rather than superposing many meanings, unlike dense Transformer weight matrices.
- **Practical variant:** low-rank factorization of this synaptic update yields **BDH-GPU**, which is efficient enough to train on standard accelerators while matching Transformer-like scaling behavior on language and translation benchmarks.

BDH-CQ (arXiv:2608.09888, "*In-Context Learning with Recurrent Latent Reasoning*") builds on BDH to remove verbalized chain-of-thought entirely. It has two recurrent components: a **persistent contextual memory** that is updated as in-context demonstrations stream in, and a separate **high-dimensional workspace** that is iteratively transformed to solve the current query — reasoning happens as repeated computation inside a continuous latent state, never projected into vocabulary tokens. Pathway reports a 150M-parameter BDH-CQ model scoring 29.5% pass@2 on ARC-AGI-1 at roughly $0.0007/task, about 11× cheaper per task than GPT-5.6 Luna (low reasoning effort), which is the headline efficiency argument for latent-over-verbalized reasoning.

> **Why this matters for the pedagogy:** the single sharpest teaching hook is that BDH-CQ never emits an intermediate sentence — a learner can watch the *same* latent state get refined turn-by-turn and directly compare it to a CoT model that must externalize each step, making the "compute happens in latent space, not in text" claim something a learner can literally see rather than take on faith.

2. Candidate concept review — which has the sharpest interactive potential

Candidate concept	Interactive potential assessment
Linear Attention vs. Softmax Attention	Strong but well-worn territory (many existing explorables already visualize O(n) vs $O(n^2)$ attention). Harder to make the BDH linkage feel load-bearing rather than bolted on, since linear attention isn't BDH's core novelty.
Test-Time Adaptation: HRM/TRM vs. BDH-CQ	Rich but heavier to build — comparing two different recurrent-reasoning families needs two toy models running side by side, which raises implementation risk inside a short sprint.
Latent-Space Reasoning vs. Verbalized CoT	**Sharpest.** A single small recurrent latent-reasoning toy model (BDH-CQ-flavored) can be run entirely client-side, its hidden state visualized as it converges over K recurrence steps, and directly contrasted against a token-by-token CoT trace on the identical toy puzzle — one substrate, one clear falsifiable claim, and a natural slider (recurrence depth K) that changes the visualization live.

Selected concept: Latent-Space Reasoning vs. Verbalized Chain-of-Thought, with the BDH module grounding the toy model's update rule in BDH's Hebbian/sparse formalism.

3. Interactive web architecture for <1s client-side feedback

- **Compute substrate:** a tiny (2–4 layer, <1M parameter) latent-reasoning network implemented directly in TypeScript/WASM (hand-rolled matrix ops or a minimal WASM-compiled numeric kernel) rather than a full ML framework in the browser — the model is small enough that a framework's load time would dominate the <1s budget. Pyodide/ONNX Runtime Web are viable fallbacks only if the toy model needs to be trained/exported from Python and then just replayed; for a hand-built demo model, hand-rolled WASM/TS keeps cold-start near-zero.
- **State & rendering:** React for controls/layout, an observable/signal-based state store (e.g., a small reactive store, not Redux ceremony) so every slider change triggers a synchronous recompute-and-repaint loop, and Canvas (not SVG/DOM) for the hidden-state heatmap and token-probability bars, since Canvas repaints at slider-drag speed without DOM thrashing.
- **Feedback loop:** slider change → recompute forward pass in-thread (model is small enough to skip a Web Worker; if it grows, offload to a Worker to keep the main thread's input latency low) → repaint hidden-state trajectory and ground-truth-vs-estimate overlay — all synchronous, no network round-trip, which is what actually guarantees sub-1s "truth-beside-estimate" feedback rather than just making it likely.
- **No backend GPU dependency:** the entire judged interaction must run from a static, zero-login web build (per the submission constraint), so the architecture deliberately avoids any server-side inference call on the interaction path — only static asset hosting is required.

PROMPT 2 · TRACK SELECTION

Track Selection — Scoring, Risk, and Recommendation

1. Side-by-side scoring breakdown

Dimension	Track A — Rime Voice-Native	Track B — Pathway BDH Explainer
Implementation velocity	Medium. LiveKit + Rime plugin scaffolding is fast to stand up, but a *correct* full-duplex interruption handler is genuinely new engineering, not boilerplate.	Medium-High. A tiny hand-built latent-reasoning toy model and Canvas visualization is a well-bounded, single-repo frontend build with no external service dependency.
Debugging difficulty	High. Race conditions across WebSocket audio streams, VAD timing, and async tool calls are notoriously non-deterministic and hard to reproduce.	Low-Medium. Pure client-side numeric code with deterministic seeds is trivially reproducible and debuggable with normal breakpoints/console logging.
Risk of demo failure during judging	High. Live audio over judges' network/mic conditions, third-party API latency (STT/LLM/Rime), and telephony edge cases are all live-demo hazards.	Low. A static, zero-login web build removes network dependency, live mic dependency, and third-party API uptime risk entirely.
Tooling maturity	High for the happy path (LiveKit Agents + Rime plugin are documented, maintained integrations); low for the specific interruption/cancellation edge cases, which have few off-the-shelf examples.	Low-Medium. No off-the-shelf "toy BDH-CQ in the browser" library exists; the team is building the pedagogical substrate from the paper's equations, which is more novel work but also more open-ended.
Evaluation subjectivity	Lower. TTFA and cancellation-latency thresholds are numeric pass/fail, which favors teams with clean evidence.	Higher. "Learning effectiveness," "ownership & live defense," and "craft/provenance" are judged qualitatively and reward strong live explanation skills.

2. Failure modes & blockers

Track A — hardest engineering hurdles

- **Audio pipeline drift:** STT partials arriving out of order relative to TTS playback state can desynchronize the "what has the user actually heard" bookkeeping, corrupting the transcript used for the next LLM turn.
- **WebSocket race conditions on barge-in:** a cancel signal and an in-flight Rime audio chunk can cross on the wire; without a monotonic turn/sequence id on every payload, a stale chunk can play *after* cancellation and silently fail the <200ms assertion.

- **Latency budget over telephony vs. web:** PSTN/SIP trunking adds codec transcoding and jitter buffer delay that a WebRTC browser demo won't show, so a TTFA measured only in-browser can be non-representative of the dispatcher persona's real channel.
- **Edge-case pronunciation failures:** domain jargon, part numbers, or multilingual caller names can break TTS prosody in ways that only show up under load-bearing content, not in a generic demo script.

Track B — pedagogical and technical pitfalls

- **Scripted toy instead of a reactive substrate:** the single biggest risk — pre-baking animation keyframes instead of running real forward passes fails "Interactive substrate" (15%) outright and is easy for judges to catch by dragging a slider to an unscripted value.
- **Overcomplicated math losing the learner:** exposing the full BDH derivation without a guided on-ramp can tank "Learning effectiveness" (15%) even when the underlying model is correct.
- **Weak/bolted-on BDH linkage:** treating BDH-CQ as a footnote instead of the mechanism the toy model actually implements directly costs "BDH integration" (10%) and reads as compliance rather than genuine engagement.
- **Math/algorithmic inaccuracies:** "Technical correctness & depth" is 25% of the rubric — the single largest line item in either track — so an incorrect update rule or a mislabeled axis is disproportionately costly.

3. Decision tree

> **Choose Track A** if your team has: (1) real WebRTC/telephony or LiveKit experience already, (2) someone comfortable debugging async race conditions under time pressure, and (3) confidence you can rehearse a live/recorded demo enough times to trust it in front of judges.

> **Choose Track B** if your team has: (1) ML/math depth strong enough to correctly implement and explain a Hebbian/sparse update rule, (2) frontend/visualization skill to make a live Canvas substrate feel alive rather than static, and (3) at least one member ready for rigorous live technical defense of the claim and the math.

4. Final recommendation

Recommendation: Track B (Pathway BDH Explainer) has the higher ceiling for a short, judged sprint — conditional on real ML/math depth on the team.

Three reasons drive this, in order of weight: (1) **demo-day risk asymmetry** — Track B's zero-login static build removes the single largest hackathon failure mode (live third-party API/network/mic dependency during judging), while Track A's score is directly exposed to exactly that risk; (2) **rubric concentration** — Track B puts 25% on technical correctness/depth and 15% on interactive substrate, both of which a strong team can fully control through preparation and testing, whereas Track A's 25% "hard voice engineering" line requires solving genuinely open race-condition-prone systems problems under time pressure; (3) **reproducibility** — a deterministic client-side toy model is far easier to rehearse to a polished, repeatable state than a live full-duplex voice interruption demo. Track A remains the right call specifically if the team already has proven WebRTC/telephony chops and wants to compete on an engineering feat few teams will attempt correctly — that differentiation is real, but it is a bet on execution risk, not a hedge against it.

PROMPT 3 · SOLUTION ASSEMBLY

Approach 1 — Track A Solution Blueprint (Rime Voice-Native)

1. Product concept & necessity of voice

"FieldVox" — a hands-busy industrial safety inspection copilot. An inspector wearing gloves and PPE walks a facility, narrates findings aloud, and the agent logs structured defect records, reads back spec thresholds on request, and confirms each checkpoint — all without the inspector touching a device. A screen fails here specifically because PPE and physical contact with equipment make typing either unsafe or impossible.

2. Selected voice engineering challenge

Full-duplex interruption & conversation continuity during long-running tool calls. When the inspector asks "what's the torque spec for this valve," the agent must query a spec database (a slow tool call) while remaining interruptible — if the inspector immediately follows with "actually, skip that, log a crack on the housing instead," the agent must cancel the in-flight lookup, discard any queued audio for the abandoned answer, and pivot cleanly.

3. System architecture & tech stack

Pipeline: LiveKit Agents (session orchestration) → Silero VAD (continuous) → Deepgram/Whisper streaming STT → streaming LLM orchestrator with function-calling fences (tool calls emitted as structured deltas the orchestrator can intercept mid-stream) → Rime TTS over WebSocket (`use_websocket=True`) for low-latency, word-timestamped audio.

State machine

```
[LISTENING]
   │ VAD speech-end + STT final transcript
   ▼
[THINKING] ──(LLM emits normal reply)──▶ [SPEAKING] ──(reply fully played)──▶ [LISTENING]
   │
   │ LLM emits function-call fence (slow tool call, e.g. spec lookup)
   ▼
[TOOL_PENDING] (agent may speak a short "let me check that" filler while awaiting the
tool)
   │
   ├─ tool resolves normally → [THINKING] (re-prompt LLM with tool result) → [SPEAKING]
   │
   └─ VAD fires during TOOL_PENDING or SPEAKING (barge-in)
        │
        ▼
   [INTERRUPTED]
        │  1. cancel in-flight tool call (abort controller)
        │  2. purge queued Rime WebSocket audio frames for the active turn id
        │  3. mark active turn "aborted" in transcript state
        │  4. discard any LLM continuation tokens still streaming
```

```
        ▼
    [LISTENING] (fresh turn begins from the new user utterance)
```

4. Acceptance test specification (`RIME_EVIDENCE.md`)

```
## RIME_EVIDENCE.md — reproducible acceptance tests

### Preflight
1. Verify LIVEKIT_URL, LIVEKIT_API_KEY, LIVEKIT_API_SECRET, RIME_API_KEY are set.
2. `python preflight.py` — opens a LiveKit room + a Rime WebSocket session,
   asserts both handshake within 2s, exits non-zero on failure.

### Test 1 — Sub-400ms TTFA (happy path)
1. Inject a pre-recorded WAV utterance ("what's the torque spec for bolt A12")
   via a synthetic LiveKit participant.
2. Log timestamps: vad_speech_end, stt_final, llm_first_token, tts_first_byte,
   track_publish_first_frame.
3. Repeat 20x. Assert p50 TTFA < 400ms and p90 TTFA < 550ms.
   FAIL the build if either threshold is missed.

### Test 2 — Interruption during a slow tool call
1. Register a mock `spec_lookup` tool with an injected 3000ms artificial delay.
2. Inject utterance A: "what's the torque spec for the flange."
   → assert agent enters TOOL_PENDING within 300ms of the tool call being emitted.
3. At t+800ms (while TOOL_PENDING, tool still unresolved), inject utterance B:
   "actually skip that, log a crack on the housing instead."
4. Assert:
   a. Last audio frame published to the track occurs within 200ms of utterance B's
      VAD-fire timestamp.
   b. The mock tool's abort signal fires (tool call is actually cancelled, not just
ignored).
   c. Zero Rime audio frames tagged with turn_id(A) arrive after the cancel timestamp
      (stale payloads discarded).
   d. Turn A is recorded as `status: aborted` in the session transcript log.
   e. Turn B is processed as a fresh, independent turn (state rollback confirmed).
5. Repeat 10x with randomized injection offsets (400ms–1800ms) to rule out a
   timing-specific pass.

### Reporting
Emit results as `evidence/run-.json` with per-test pass/fail, p50/p90
TTFA, and the cancellation-latency distribution across all runs.
```

5. 4-minute demo script

Segment	Time	Beat
Problem pitch	0:00–0:30	Show the inspector's gloved hands, state plainly: "a screen fails here — this has to be voice."
Normal flow	0:30–2:00	Live walk-through: inspector asks a spec question, agent answers via Rime TTS with visible TTFA counter on screen (<400ms).
Stress / interruption test	2:00–3:30	Trigger the slow tool-call scenario live, interrupt mid-answer, show the agent cancel cleanly and pivot — the centerpiece of the "hard voice engineering" claim.
Metric / evidence display	3:30–4:00	Cut to the `RIME_EVIDENCE.md` test run output: p50/p90 TTFA and cancellation-latency numbers on screen as proof, not just narration.

Approach 2 — Track B Solution Blueprint (Pathway Frontier Explainer)

1. Concept & falsifiable claim

Concept: Recurrent Latent Reasoning vs. Chain-of-Thought.

> *Falsifiable claim: "Latent recurrent refinement (BDH-CQ-style) increases puzzle-solving accuracy over fixed token budgets without emitting intermediate reasoning tokens, but degrades relative to verbalized CoT once the puzzle requires holding more than K discrete constraints simultaneously in the latent workspace."*

This is falsifiable because the toy demo can be pushed past its own claimed regime live: increase the constraint count slider until the latent model's accuracy visibly crosses below the CoT baseline, which is the exact predicted failure boundary rather than a hand-waved limitation.

2. Interactive substrate design

- **Toy system:** a 2-layer BDH-CQ-flavored recurrent network (hidden width ~64, non-negative sparse activations) implemented in TypeScript, compiled/executed directly in-browser — small enough to run at 60fps without WebAssembly, though a WASM numeric core is a safe upgrade path if profiling shows main-thread jank.
- **Controls:** recurrence depth K (1–12 steps), context length (number of in-context demonstration examples, 0–8), weight decay λ (0–0.1), and a "constraint count" slider driving the falsifiable claim's stress test.
- **Visualization:** (a) a hidden-state heatmap redrawn on Canvas after every recurrence step, showing the latent workspace visibly converging (or failing to) over K steps; (b) a live output token-probability bar chart plotted against ground truth, updated the instant a slider moves; (c) a side-by-side CoT trace panel showing the same puzzle solved by an explicit step-by-step baseline, so "truth beside estimate" is a literal split-screen, not a metaphor.
- **Latency guarantee:** the whole forward pass for this toy size is sub-millisecond on any judge's laptop; the <1s feedback budget is dominated by React re-render, not compute, so state updates are batched to a single repaint per slider tick.

3. The BDH / BDH-CQ module (mathematical exposition)

- **From attention to synaptic memory:** standard attention retrieves value vectors weighted by query-key similarity; BDH reframes this retrieval as a Hebbian-updated synaptic memory over sparse neuron activations, so "attention weight" becomes "synaptic strength between co-active units."
- **Update rule (toy-model form, matching the paper's Hebbian framing):** `W_t = W_{t-1} + η x_t y_t^T` , where x_t is the current input activation, y_t the resulting output activation, and η a learning-

rate-like plasticity constant — the toy demo's slider for λ (weight decay) is layered on top of this update to show the stability/plasticity trade-off live.

- **Non-negative sparse activations:** activations are rectified (`y = max(0, ·)`) and regularized toward sparsity, which the demo visualizes directly as most heatmap cells staying at zero — the same property the paper connects to monosemantic, interpretable synapses.
- **BDH-CQ's two recurrences:** the toy model implements both loops explicitly — a slow-updating contextual memory (rewritten once per in-context example) and a fast-updating query workspace (rewritten once per recurrence step K) — so the demo's two sliders (context length, recurrence depth) map one-to-one onto the paper's two mechanisms, keeping the BDH linkage load-bearing rather than decorative.

4. One-page concept summary blueprint (section-by-section)

Section	Content
Problem motivation	Why verbalized CoT is expensive/brittle; the cost-accuracy Pareto framing.
Technical mechanism	The two-recurrence BDH-CQ structure and the Hebbian update rule, condensed.
Comparative table	Transformer vs. Mamba vs. BDH-CQ across compute pattern, memory mechanism, and reasoning trace.
Evidence classification	Separate published benchmark results from this project's own toy-model demonstration, labeled explicitly.
Known architectural bounds	Where latent reasoning is expected to underperform CoT (the falsifiable claim's failure boundary).

The filled-in draft following this blueprint (ready to submit as the required 500–950 word PDF) is provided in the Appendix.

APPENDIX · TRACK B DELIVERABLE DRAFT

One-Page Concept Briefing (draft, 500–950 words)

Recurrent Latent Reasoning vs. Chain-of-Thought: What BDH-CQ Changes About Where Reasoning Happens

Problem motivation

Modern reasoning models solve hard problems by generating long chains of intermediate text — chain-of-thought (CoT) — before producing a final answer. This works, but it is expensive: every intermediate step must be projected through a discrete vocabulary, sampled, and re-encoded, even when the "thought" itself is really a continuous adjustment to an internal belief state. Pathway's BDH-CQ, built on the Dragon Hatchling (BDH) architecture, asks a sharper question: what if reasoning happened directly in a continuous latent space, refined over repeated computation, and never verbalized at all? On the ARC-AGI-1 benchmark, a 150-million-parameter BDH-CQ model reached 29.5% pass@2 at roughly $0.0007 per task — about eleven times cheaper per task than a comparable frontier model at low reasoning effort. This project makes that idea tangible: a small, live, in-browser model that reasons by refining a latent state, sitting next to a token-by-token CoT baseline solving the identical puzzle, so a learner can watch the difference rather than read about it.

Technical mechanism

BDH derives its architecture directly from the attention equations, but reframes retrieval as a form of synaptic memory over a sparse, non-negative population of neuron-like units, updated in a Hebbian fashion: units that fire together strengthen their connection, expressed in this project's toy model as $W_t = W_{t-1} + \eta\, x_t\, y_t^T$. BDH-CQ extends this with two nested recurrences. A slow contextual memory accumulates evidence from each in-context demonstration as it streams in. A faster query workspace is then repeatedly transformed — K recurrence steps — to converge on an answer to the current puzzle, entirely inside that continuous latent state. No intermediate step is ever decoded into a sentence; the "thinking" is the trajectory of the hidden state itself. The interactive demo exposes both recurrences as sliders (context length and recurrence depth K), so a learner can watch the hidden-state heatmap sparsify and stabilize step by step, with an output-probability-versus-ground-truth readout updating live and a CoT trace rendered in a parallel panel for direct comparison.

Comparative analysis

Architecture	Compute pattern	Memory mechanism	Reasoning trace
Transformer	Parallel attention over full context, $O(n^2)$	Dense key-value attention weights	Verbalized (CoT) or none
Mamba (SSM)	Sequential state update, linear in sequence length	Fixed-size recurrent hidden state	Typically none; single forward pass

BDH-CQ	Iterative latent refinement, K recurrence steps per query	Sparse Hebbian synaptic memory, two nested recurrences	Latent only — never verbalized

Evidence classification

Two categories of evidence are kept explicitly separate in this project. **Published evidence**: the BDH architecture (arXiv:2509.26507) and BDH-CQ (arXiv:2608.09888) papers, including the ARC-AGI-1 cost/accuracy result cited above, are Pathway's own reported benchmark figures on their production-scale model — they are cited, not reproduced here. **Demonstration evidence**: the interactive toy model in this project is a small, hand-built illustration of the *same architectural principle* (two-recurrence latent refinement over Hebbian sparse memory) running on a toy puzzle set, built to make the mechanism inspectable, not to reproduce Pathway's benchmark numbers at scale. The one-pager and the live demo both label these two evidence classes separately so neither is mistaken for the other.

Known architectural bounds

The central claim is deliberately falsifiable, not celebratory: latent recurrent refinement is expected to outperform verbalized CoT under a fixed token/compute budget when the puzzle's constraints fit within the workspace's effective capacity, but to *degrade relative to CoT* once the number of simultaneous constraints exceeds what a fixed-width latent state can hold without interference — a limit CoT does not share, since it can externalize arbitrarily many steps into text at linearly increasing cost. The demo's "constraint count" slider is built specifically to cross this boundary live, showing the latent model's accuracy fall below the CoT baseline exactly where the architecture predicts it should.

Word count target: 500–950 words (Track B hard constraint). Formatting note: for final submission, render this section alone as its own single-page PDF export.

Data Forge 2026 hackathon strategy report · generated from the three-prompt brief in `Untitled document.pdf` · sources: pathway.com/research/bdh-explainer, arXiv:2509.26507, arXiv:2608.09888, docs.livekit.io (Rime TTS plugin, adaptive interruption handling), docs.rime.ai, hackathonradar.com/database/hackathon/dataforge-2026.